AI-Powered Real Estate Property Triage System

Final Project Book — AI Engineering Course

Author: Yehuda Rokach **Project type:** Individual final project **Date:** June 2026 **Status:** Draft (skeleton — grows with the build)

Table of Contents

1. Abstract
2. Introduction & Real-World Scenario
3. System Architecture
4. Technology Choices
5. Component Design
6. Prompt Engineering
7. Results & Evaluation
8. Deployment Notes
9. Conclusions & Future Work
10. References

1. Abstract

This project designs and builds a production-style, multi-modal AI system that automates the intake and initial evaluation of real-estate property listings for a fictional agency. A listing agent submits a free-text description plus property photos; the system validates the submission, extracts structured fields, classifies each image by room type and condition, retrieves comparable past listings, and produces a ready-to-publish listing brief — then routes it to the correct team.

The system is assembled from managed and reusable building blocks rather than hand-rolled from scratch: **Amazon Bedrock Knowledge Bases** for retrieval, **Amazon Bedrock Guardrails** for safety, **Google Gemini** for all LLM generation, a **PyTorch** transfer-learning model for image analysis, **n8n** for orchestration, and a **Streamlit + Ollama** web interface.

(This abstract will be tightened once results are in.)

2. Introduction & Real-World Scenario

A real-estate agency receives dozens of new property submissions every day, each a written description plus photographs. Staff must check the submission is genuine (not spam/off-topic), identify property type/condition/features, score the uploaded images, find similar past listings, route the listing to residential vs. commercial teams, and produce a clean published brief.

This is a realistic, multi-modal workflow: it combines text understanding, image analysis, retrieval-augmented generation, safety filtering, and agent-based reasoning in one coherent product. This project automates the entire pipeline end-to-end.

Learning objectives demonstrated: deploying a multi-service AI system on cloud infrastructure; building n8n automation flows with AI nodes; a retrieval-augmented pipeline; a PyTorch image classifier; prompt engineering across five distinct surfaces; input/output safety guardrails; a multi-step reasoning agent; and a local-LLM conversational UI.

3. System Architecture

The system has four layers, each communicating with the next over HTTP.

Layer 1 — Web UI (Streamlit, local). Two working surfaces plus a monitoring extension: (a) a conversational assistant backed by a **local Ollama** server running Llama 3.1, grounded as a real-estate assistant; (b) a listing submission form that POSTs to the n8n webhook and renders the returned brief; (c) a monitoring dashboard with live processing stats.

Layer 2 — n8n orchestration. An 8-node flow: webhook trigger → guardrails input check → IF (pass/fail) → Information Extractor (Gemini) → AI Agent (Gemini; dispatches tool calls to the services) → LLM Chain (final brief) → guardrails output check → router (residential vs. commercial).

Layer 3 — FastAPI microservices. Four independent, containerised services: RAG, Image Analyser, Guardrails, and a LangGraph Agent. Each exposes a single well-defined endpoint.

Layer 4 — Managed services & external LLM. Amazon Bedrock Knowledge Bases (Titan V2 embeddings + OpenSearch Serverless) and Amazon Bedrock Guardrails; Google Gemini for all text generation; Ollama (local) for the assistant chat.

(A custom architecture diagram will be added here.)

4. Technology Choices

This project intentionally **assembles managed services** instead of hand-rolling each component, which is both the instructor's intent (a few hours of assembly) and good engineering for a solo build.

Two AWS Bedrock services (requirement: ≥2). - **Bedrock Knowledge Bases** powers the RAG service — managed vector store, embeddings, and retrieval, reusing a working pattern the author already built. - **Bedrock Guardrails** powers the Guardrails service — managed content filters, PII handling, and contextual grounding.

Gemini for all LLM generation. Rather than adding a third Bedrock dependency, all generation (RAG insight, guardrail classification/auditing, the agent's reasoning, and the n8n nodes) uses Google Gemini, which the author already uses fluently. The Gemini API key is stored in **AWS Secrets Manager** and pulled by each service using its existing AWS credentials — no secret in the repo.

Intentional deviations from the written guideline. The guideline lists NeMo Guardrails, Llama.cpp, and a local ChromaDB. We substitute **Bedrock Guardrails** (for NeMo) and **Bedrock Knowledge Bases** (for Llama.cpp + ChromaDB). This satisfies the ≥2-Bedrock-services requirement, reduces operational burden for a solo developer, and — as a side effect — already satisfies the "managed vector store" optional extension.

PyTorch transfer learning (EfficientNet-B0) for the Image Analyser, because the rubric explicitly rewards a trained model.

Component	Technology
Web UI	Streamlit
Conversational LLM	Ollama + Llama 3.1 (local)
Orchestration	n8n
LLM generation	Google Gemini

RAG	Bedrock Knowledge Base (Titan V2 + OpenSearch Serverless)
Image Analyser	PyTorch + EfficientNet-B0
Guardrails	Bedrock Guardrails (`ApplyGuardrail`) + Gemini
Agent	LangGraph + Gemini
Secrets	AWS Secrets Manager

5. Component Design

5.1 Web UI (built first)

A 3-tab Streamlit app. **Assistant** tab streams chat to the local Ollama `/api/chat` endpoint with a real-estate system prompt that refuses off-topic questions, gives no legal advice, and invents no prices (this is prompt surface #5). **Submit Listing** tab posts the description, image URLs, and agent name to the n8n webhook and renders the returned brief, image condition scores, and similar listings. **Dashboard** tab shows live stats from a local event log. During early development the submit tab is tested against a mock brief so the whole UI is demoable before n8n exists.

5.2 RAG service (`POST /query`)

Embeds the description and retrieves the top-3 most similar past listings from a Bedrock Knowledge Base pre-populated with ≥20 synthetic listings, then generates a short insight with Gemini that cites the listing it drew from and never fabricates facts. Output: `{ similar_listings, insight }`.

5.3 Image Analyser (`POST /analyse`)

A transfer-learning EfficientNet-B0 with two heads: room type (kitchen, bathroom, living room, bedroom, exterior, other) and a 1–5 condition score. Below a confidence threshold it returns `uncertain`. Output: `{ room_type, condition_score, confidence }`. Trained on a public Kaggle room dataset; condition labels bootstrapped with Gemini Vision and hand-corrected.

5.4 Guardrails service (`POST /check/input`, `POST /check/output`)

Input check: Bedrock `ApplyGuardrail` for safety/PII plus a Gemini classifier that accepts only genuine property listings in the expected language (rejecting others, including unexpected languages — the multilingual extension). Output check: PII masking plus a Gemini factuality-vs-source check that catches invented prices, fabricated certifications, and false legal

claims. Output: `{ pass, reason, safe_text }`.

5.5 LangGraph Agent (`POST /agent/run`)

A 3-node state graph — planner → tool executor → synthesiser — using Gemini, where the executor calls the RAG and Image Analyser services. Answers multi-step questions (e.g., "what renovation work would bring this property to condition score 5?"). Output: `{ answer, tools_used, reasoning_steps }`.

5.6 n8n flow

Eight nodes wiring the webhook through the guardrails, the Gemini Information Extractor and AI Agent, the LLM Chain that produces the brief, the output guardrail, and a router that sends residential vs. commercial listings to different teams.

6. Prompt Engineering

Five prompt surfaces are tuned and logged across at least five iterations each, with ≥10 test cases and a measured pass rate per surface: the n8n Information Extractor, the n8n AI Agent (and tool descriptions), the RAG insight prompt, the Guardrails rail prompts, and the Ollama system prompt. The full iteration history is in `property-triage-system/docs/prompt_log.md`.

(Summary of key findings to be written once the logs are complete.)

7. Results & Evaluation

To be completed after the build. - Image Analyser: test-set accuracy and confusion matrix (target >75%). - Guardrails: false-positive rate on valid listings (target <5%). - RAG: retrieval precision@3 (managed-vector-store extension benchmark). - End-to-end: sample listing → brief, with screenshots.

8. Deployment Notes

To be completed at the deployment phase. - EC2 instance type(s), ports, security-group rules. - Docker build notes (`--platform linux/amd64` on Apple Silicon). - Deviations from the specification.

9. Conclusions & Future Work

To be written at the end. Candidate future work: the human-in-the-loop feedback/active-learning loop (deferred), a managed Pinecone/Weaviate comparison, and richer monitoring.

10. References

- Project guideline: *AI Engineering — Final Project: AI-Powered Real Estate Property Triage System.*
- Amazon Bedrock Knowledge Bases, Amazon Bedrock Guardrails (AWS documentation).
- Google Gemini API documentation.
- PyTorch transfer-learning tutorial; torchvision model zoo.
- n8n documentation; Ollama documentation; Streamlit documentation.